

LGTM OBSERVABILITY STACK PROJECT

통합 관측성 구축 프로젝트 결과 발표

Loki · Grafana · Mimir 통합 파이프라인 수립

📅 수행 기간

2026. 06. 08. ~ 06. 28. (3주)

☑ 배포 환경

Cloud VM (Ubuntu 22.04 LTS)

🏗 오케스트레이션

Docker Compose 기반

📁 산출물 레포지토리

github.com/kinx12399/LGTM_Observability_stack

발표 목차 (Agenda)

01 | 프로젝트 개요

02 | 아키텍처 및 파이프라인 구성

03 | 서비스 구동 검증 및 테스트

04 | 프로젝트 고도화 내역

05 | 트러블슈팅

06 | 프로젝트 회고

07 | 향후 시스템 발전 계획

08 | Q&A

01. 프로젝트 개요

1.1 프로젝트 개요



프로젝트 목적

클라우드 VM 환경에서 분산된 관측성 구성 요소인 **Loki** · **Grafana** · **Mimir**를 직접 단일 통합 스택으로 빌드합니다.

이를 통해 로그 · 메트릭 통합 수집 및 통합 시각화 파이프라인을 완성하는 것을 목표로 합니다.



수집 스택 범위

- Loki: 고성능 초경량 로그 수집
- Mimir & Prometheus: 메트릭 데이터 장기 보관
- Grafana: 통합 웹 콘솔 시각화
- Tempo: 심화 과정인 트레이스 연동



수행 형태 및 배포

단독 수행(1인) 형태로 기획, 설계, 아키텍처링, 트러블슈팅까지 전과정을 수행하였습니다.

Docker Compose 기반의 가상 브리지 네트워크 구조를 적용하여 안전한 고립 배포를 완수했습니다.

1.2 하드웨어 스펙 및 가상머신 사양

하드웨어 리소스 항목	최소 사양 (Minimum)	권장 사양 (Recommended)	실제 프로젝트 배포 스펙
vCPU Cores	2 Core	4 Core	4 Core (IXcloud 가상 머신)
System RAM	4 GB	8 GB	8 GB (OOM 방지를 위한 필수 스펙)
SSD Storage	30 GB SSD	50 GB SSD	50 GB SSD (MinIO Object Storage 볼륨)
Operating System		Ubuntu 22.04 LTS (64-bit)	Ubuntu 22.04 LTS 동일 환경

 메모리 경고: RAM 4GB 미만 환경에서는 Mimir 청크 분산 버퍼 캐싱 및 분산 컴포넌트 기동 시 Out of Memory(OOM)가 발생할 가능성이 대단히 높으므로 가상 머신 사양을 반드시 8GB 이상으로 권장 확보했습니다.

1.3 소프트웨어 및 인프라 보안 포트 정의

필수 소프트웨어 규격

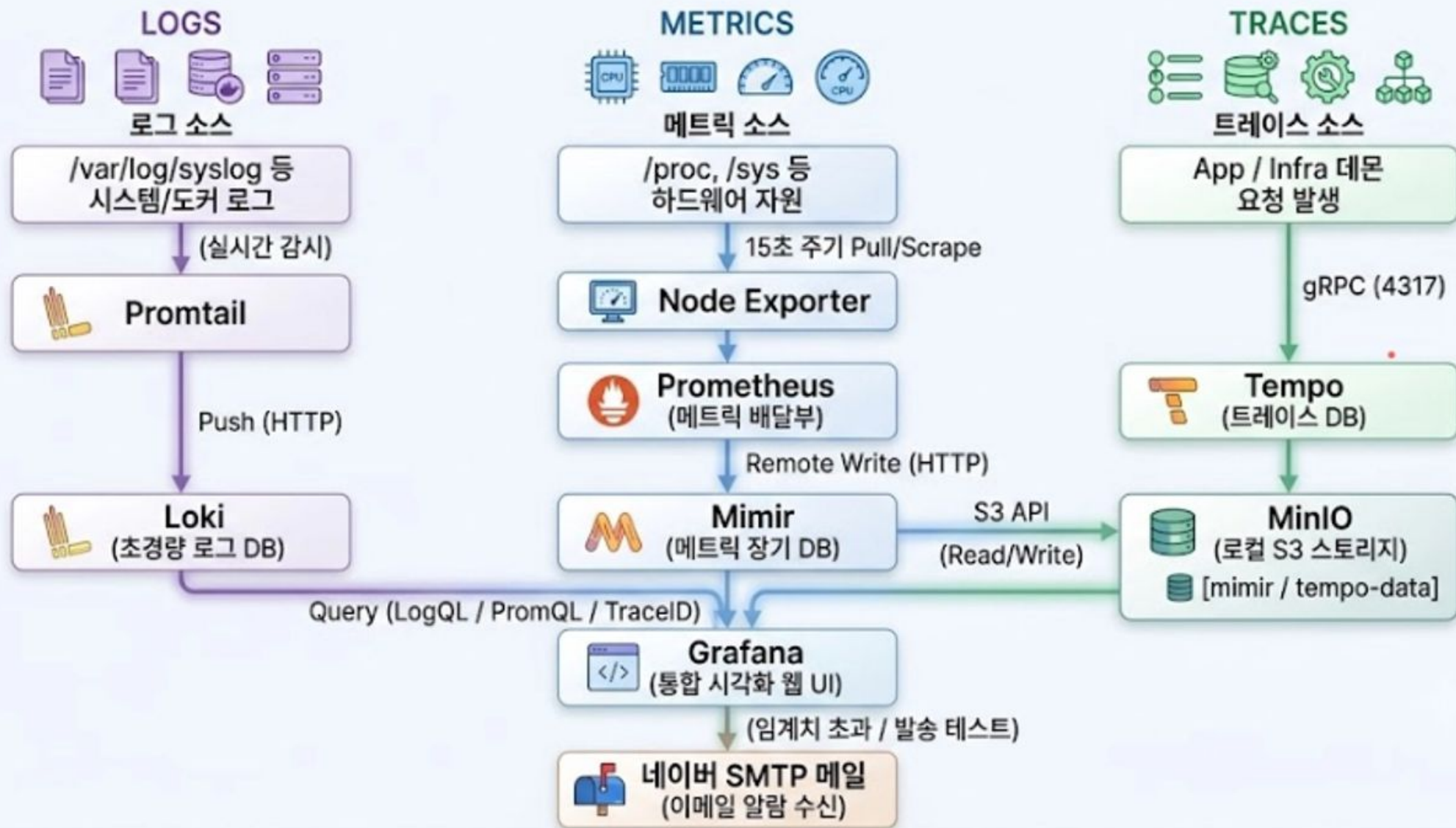
명칭	버전	용도
Docker Engine	26.x+	컨테이너 격리 런타임
Docker Compose	v2.x	컴포즈 v2 플러그인
Prometheus	2.x 이상	메트릭 수집 → Mimir Remote Write 설정
Promtail	최신 동일	로그 에이전트 및 전송 수집기

내부 포트 보안 격리 정책

컴포넌트	포트	보안 정책 및 접근 제한
Grafana UI	3000/TCP	외부 허용 (웹 콘솔 접속)
Loki Server	3100/TCP	내부망 전용 (외부 오픈 금지)
Mimir API	9009/tcp	내부 Docker 통신 전용
Node Exporter	9100/TCP	로컬 수집 한정 비개방

02. 아키텍처 및 파이프라인

2.1 통합 관측성 (Observability) 아키텍처



2.1 통합 관측성 (Observability) 아키텍처



Promtail & Loki 상세 구성

Promtail 주요 설정 (Log Agent)

- 수집 대상: 시스템 및 앱 로그 (/var/log/*log)
- 로그 전송: http://loki:3100 Push 방식
- 보안로그 라벨링:
 - login, logout, auth 키워드 포함 시 security
 - 그 외 일반 로그는 generic 라벨 지정
- 조건부 데이터 드롭:
 - label="generic" && level="INFO"인 경우 스토리지 절약을 위해 자동 폐기

Loki 주요 설정 (Log DB)

- Ingestor 정책:
 - 수집 후 1시간 경과 시 또는 1MB 도달 까지 Chunk 기록
- Limits Config:
 - 1년 이상의 과거 데이터 인입 거부 (Data Integrity)
 - 일반 로그 보관: 30일 (Retention)
- 보안 로그 특례:
 - {log_type="security"} 데이터는 1년간 장기 보관
- Compactor:
 - 활성화 상태, 삭제 대상 확정 후 2시간 유예 후 영구 삭제

2.1 통합 관측성 (Observability) 아키텍처



Promtail & Loki 상세 구성

```
2026-06-15 16:29:25.761 ERROR Jun 15 16:29:25 localhost app-payment: {"time
2026-06-15 16:29:25.760 INFO Jun 15 16:29:25 localhost app-order: {"time
2026-06-15 16:29:25.760 INFO Jun 15 16:29:25 localhost app-auth: {"time
2026-06-15 16:29:25.760 ERROR Jun 15 16:29:25 localhost app-payment: {"time
2026-06-15 16:29:25.760 ERROR Jun 15 16:29:25 localhost app-db: {"timestamp
2026-06-15 16:29:25.760 WARN Jun 15 16:29:25 localhost app-auth: {"time
2026-06-15 16:29:25.760 WARN Jun 15 16:29:25 localhost app-db: {"timestamp
2026-06-15 16:29:25.760 INFO Jun 15 16:29:25 localhost app-payment: {"time
2026-06-15 16:29:25.760 INFO Jun 15 16:29:25 localhost app-auth: {"time
```

```
2026-06-19 15:02:22.875 ERROR Jun 19 15:02:22 localhost app-payment: {"t
2026-06-19 15:02:22.874 INFO Jun 19 15:02:22 localhost app-auth: {"time
2026-06-19 15:02:22.874 ERROR Jun 19 15:02:22 localhost app-payment: {"t
2026-06-19 15:02:22.874 ERROR Jun 19 15:02:22 localhost app-db: {"timest
2026-06-19 15:02:22.874 WARN Jun 19 15:02:22 localhost app-auth: {"time
2026-06-19 15:02:22.874 WARN Jun 19 15:02:22 localhost app-db: {"timest
2026-06-19 15:02:22.874 INFO Jun 19 15:02:22 localhost app-auth: {"time
```

2.1 통합 관측성 (Observability) 아키텍처

Prometheus

Prometheus 주요 설정 (TSDB)

- **global:** 메트릭 수집 주기 15초 마다 데이터 Scrape
- **Scrape_configs:**
 - Prometheus 자기 자신 (localhost:9090)
 - Node Exporter (node-exporter:9100) 수집
- **Remote_write:**
 - Mimir 엔드포인트로 데이터 전송
<http://mimir:9009/api/v1/push>

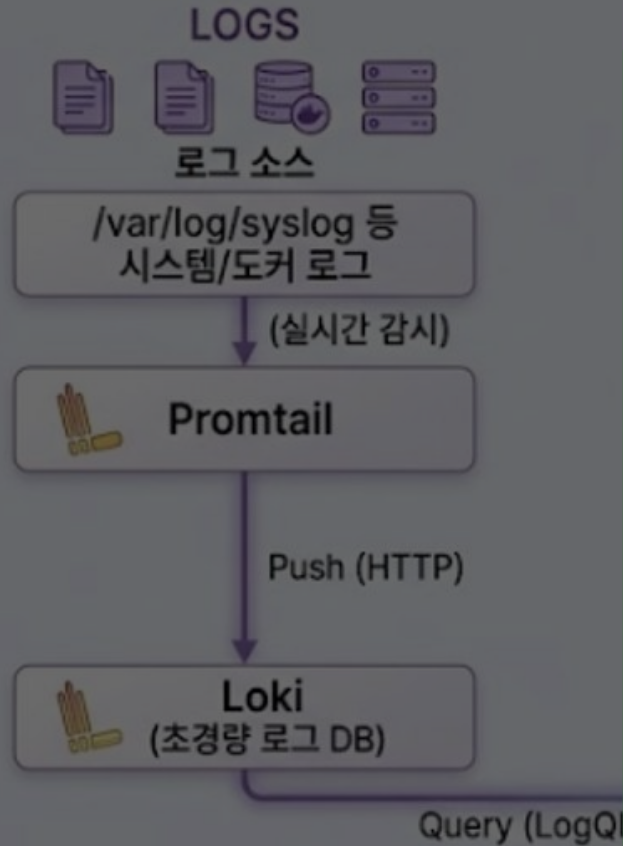


Mimir

Mimir 주요 설정 (Metric DB)

- **실행모드 (Monolithic):**
Ingester, Querier, Compactor 등 모든 요소를 하나의 프로세스로 통합
- **Limits 정책:**
메트릭 데이터 보관기간 30일 설정
- **Compactor:**
 - 활성화 상태
 - 메트릭 블록 압축 및 병합
 - 보관 만료 데이터 블록 자동 삭제

2.1 통합 관측성 (Observability) 아키텍처



METRICS

Tempo

Tempo 주요 설정 (Trace DB)

- **Distributor:**
 - OpenTelemetry Protocol(OTLP) 포맷 설정
 - gRPC 통신 방식 선택
- **Storage:**
 - Minio 연결
 - WAL(write-ahead log)로 로컬디스크 임시보관
- **Backend_worker:**
 - 3일 보관, 1시간 단위 데이터 압축
 - 만료 데이터 1시간 보관 후 삭제

(이메일 알람 수신)

TRACES



트레이스 소스

App / Infra 데몬
요청 발생

gRPC (4317)

Tempo
(트레이스 DB)

MinIO
(로컬 S3 스토리지)
[mimir / tempo-data]

03. 서비스 구동 검증 및 테스트

3.1 전체 서비스 컨테이너 구동 검증



7개 핵심 통합 프로세스 기동 완료 grafana, loki, promtail, prometheus, mimir, tempo, minio의 모든 컨테이너가 `docker compose ps` 상에서 running 상태를 정상적으로 충족함을 완수했습니다.



컴포즈 오케스트레이션 역등성 `docker compose up -d` 단일 커맨드 스크립트를 통해 전체 컨테이너가 순차적으로 상호 디펜던시 및 헬스 체크 상태를 보장하며 성공적으로 인스턴싱됩니다.

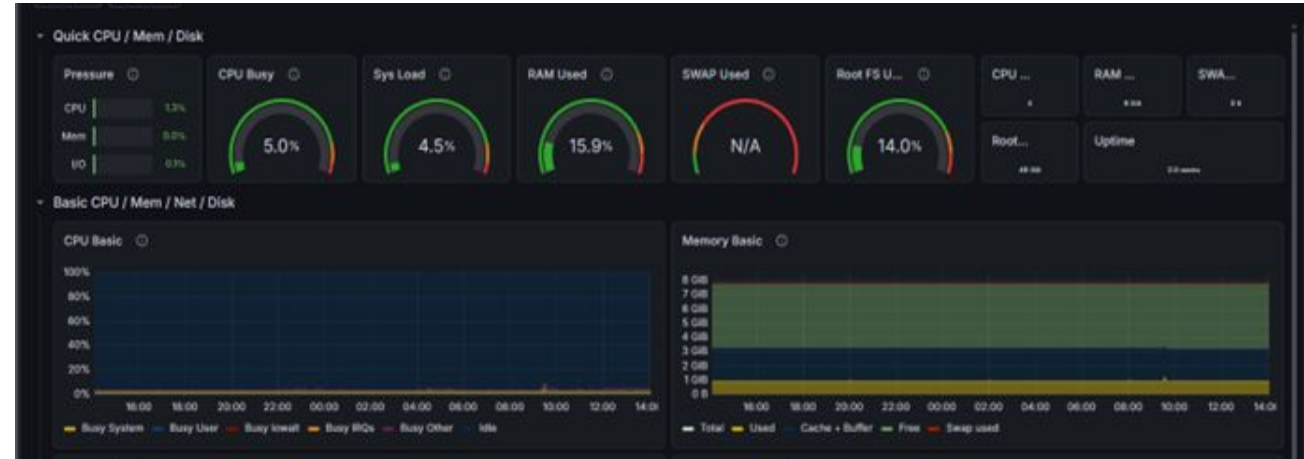
```
stack$ docker compose ps
COMMAND                SERVICE        CREATED        STATUS
/run.sh                grafana        4 days ago    Up 4 days
/usr/bin/loki -conf... loki           4 days ago    Up 4 days
/bin/mimir -config... mimir         4 days ago    Up 4 days
/usr/bin/docker-ent... minio           4 days ago    Up 4 days (healthy)
/bin/node_exporter ... node-exporter  4 days ago    Up 4 days
/bin/prometheus --c... prometheus     4 days ago    Up 4 days
/usr/bin/promtail -... promtail       4 days ago    Up 4 days
/tempo -config.file... tempo          4 days ago    Up 4 days
```


3.2 Grafana 통합 대시보드 시각화

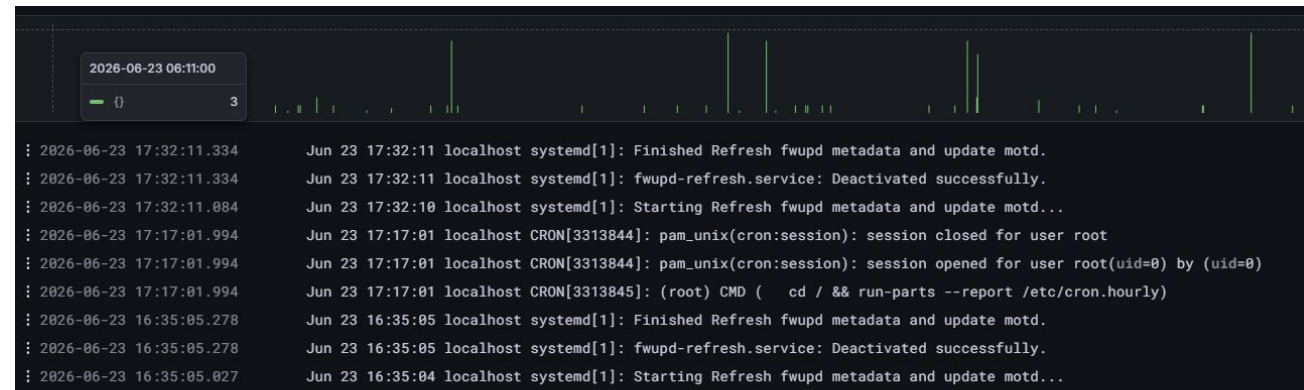
<http://1.201.177.179:3000/>



Node Exporter 메트릭 리포팅 Mimir로부터 유입되는 호스트 서버의 리소스를 집계하여 CPU 사용률, RAM 여유분, Swap, 디스크 가용 용량, 네트워크 트래픽 basic 인터페이스를 1분 해상도로 완전 표출합니다.



LogQL 기반 통합 로그 검증 Promtail이 실시간 추적(Tailing) 중인 호스트 시스템 로그 중 보안 이벤트 및 커널 로그들을 필터링 검색하여 Grafana 메트릭 타임라인과 완벽 연동합니다.

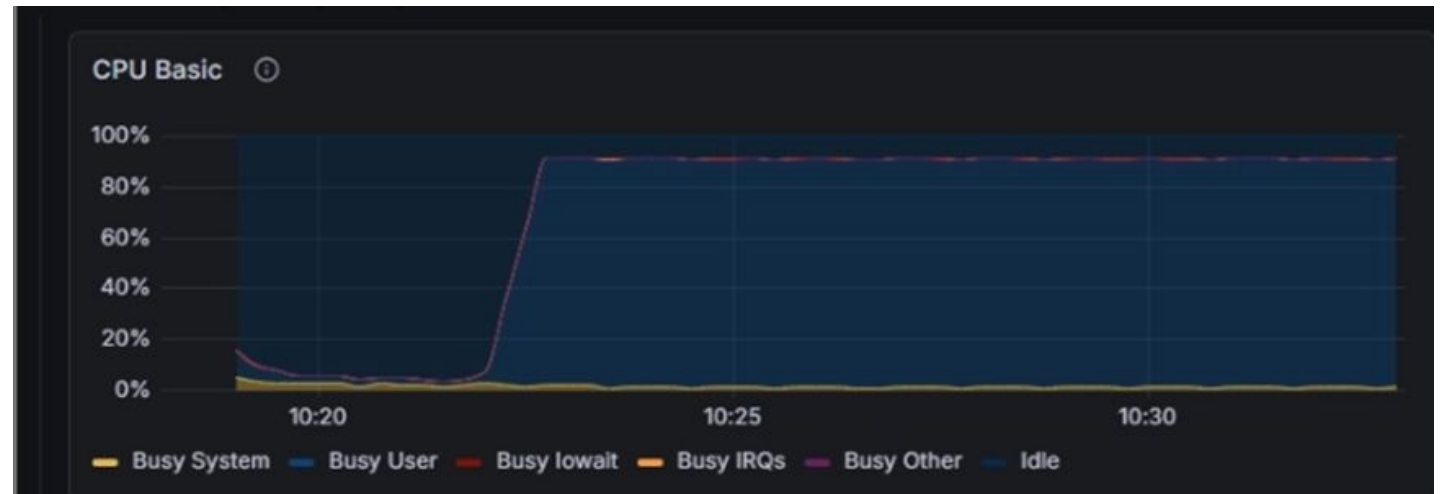


3.3.1 CPU 부하 임계치 알람 설정 및 검증

알람 설정 임계치

CPU Usage Thresholds

- 1. CPU 사용율 5분 평균값 > 80%
(5분 지속)
- 2. CPU 사용율 1분 평균값 > 80%
(1분 지속)
- 3. CPU 사용율 > 80%
(5분 지속)



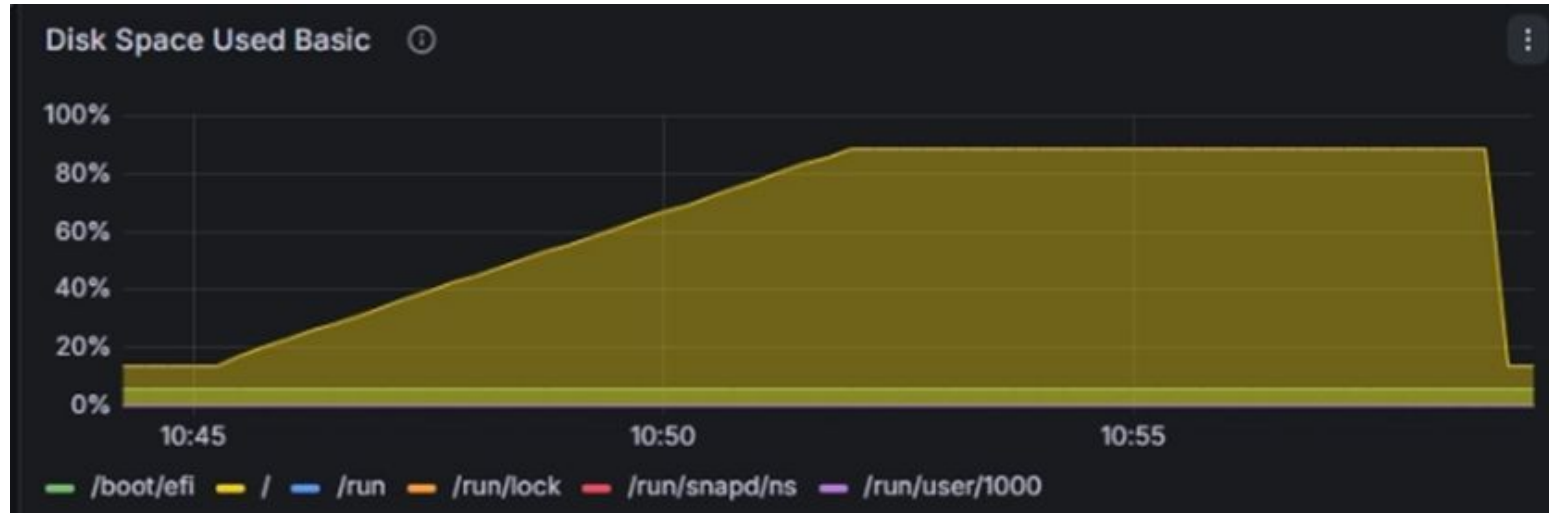
✉	LGTM-Alert-Bot	[RESOLVED] [Warning] High CPU Usage(1분평균) Monitoring 🔍 ↗	1
✉	LGTM-Alert-Bot	[RESOLVED] [Warning] High CPU Usage(5분 지속) Monitoring 🔍 ↗	1
✉	LGTM-Alert-Bot	[RESOLVED] [Warning] High CPU Usage(5분평균) Monitoring 🔍 ↗	1
✉	LGTM-Alert-Bot	[FIRING:1] [Warning] High CPU Usage(5분평균) Monitoring 🔍 ↗	1
✉	LGTM-Alert-Bot	[FIRING:1] [Warning] High CPU Usage(5분 지속) Monitoring 🔍 ↗	1
✉	LGTM-Alert-Bot	[FIRING:1] [Warning] High CPU Usage(1분평균) Monitoring 🔍 ↗	1

3.3.2 DISK 부하 임계치 알람 설정 및 검증

알람 설정 임계치

DISK Usage Thresholds

- 1. DISK 사용율 > 85% 상회 즉시
- 2. DISK 사용율 > 85% 5분 지속 시



☐	☐	☐	LGTM-Alert-Bot	[RESOLVED] [Critical] High Disk Usage(즉시 알람) Monitoring (/dev/vda1 ext4 node-exporter:9100 node-exporter /)	오전 11:02
☐	☐	☐	LGTM-Alert-Bot	[RESOLVED] [Warning] High Disk Usage Monitoring (/dev/vda1 ext4 node-exporter:9100 node-exporter /)	오전 11:02
☐	☐	☐	LGTM-Alert-Bot	[FIRING:1] [Warning] High Disk Usage Monitoring (/dev/vda1 ext4 node-exporter:9100 node-exporter /)	오전 10:57
☐	☐	☐	LGTM-Alert-Bot	[FIRING:1] [Critical] High Disk Usage(즉시 알람) Monitoring (/dev/vda1 ext4 node-exporter:9100 node-exporter /)	오전 10:52

04. 프로젝트 고도화

4.1 고도화 ①: 이미지 태그 버전 고정 (Pinning)



배경: latest 태그 지양

편의를 위해 설정된 `:latest` 도커 태그는 배포 시점마다 다른 기술 스택 버전을 내려받아, 예기치 않은 문법 및 파싱 고장 장애의 원인이 됩니다.



조치: 실행 컨테이너 버전 역분석

도커 허브의 :latest 태그를 찾거나, 구동 중인 컨테이너의 메타데이터를 역추적하여 Grafana `13.0`, Loki `3.7.2`, Mimir `3.1.0` 버전을 도출해 고정했습니다.



효과: 배포 인프라 먹등성 확보

인프라 환경의 일관성 및 재현성을 보장하고, 버전 불일치로 인해 발생하는 장애원인을 제거했습니다.

```
ubuntu@vm-project1-lgtm:~/LGTM_Observerbility
NAME                IMAGE
grafana              grafana/grafana:13.0
loki                 grafana/loki:3.7.2
mimir              grafana/mimir:3.1.0
minio                minio/minio
node-exporter        prom/node-exporter:v1.11.1
prometheus           prom/prometheus:v3.12.0
promtail             grafana/promtail:3.6.8
tempo                grafana/tempo:3.0.0
```

4.2 고도화 ②: Grafana Provisioning (IaC)

배경: UI 수동 마우스 클릭 반복의 비효율

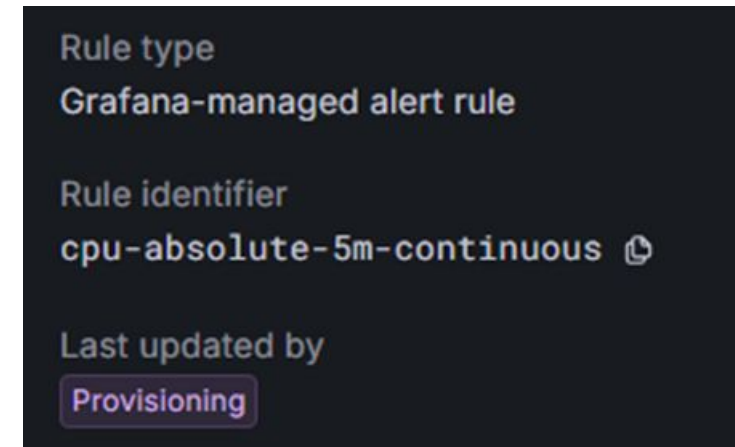
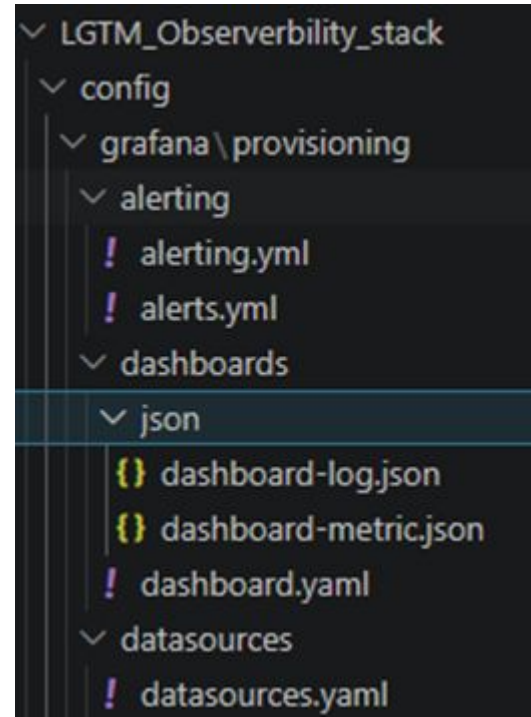
대시보드와 데이터소스를 이관할 때, 매번 수동 마우스 조작 설정 방식은 실수 위험이 크고 재배포 시 복구 지연을 초래

조치: 데이터소스/대시보드 코드화 (IaC)

Grafana Provisioning 시스템 디렉터리 구조를 구현하여 Mimir, Loki, Tempo 데이터소스 연동 파일(YAML) 및 대시보드 템플릿(JSON)을 파일화

효과: 복구 즉시성 및 Git 추적성 보장

도커 컴포즈 최초 구동 시 어떠한 수동 마우스 작업 없이 완벽히 로드되며, 설정 변경 사항을 Git 커밋 이력으로 상세 관리



4.3 고도화 ③: 컨테이너 최소 권한 원칙 (Least Privilege)



배경: Tempo의 Root 구동 취약성 발견

Tempo 기동 시 로컬 호스트 마운트 디렉터리의 읽기/쓰기 권한 에러 해결을 위해 손쉽게 `user: "root"` 설정을 사용해 보안 위험이 존재



조치: 초기화 컨테이너 기반 권한 분리

`tempo-init` 컨테이너에서 디렉토리 소유권 수정을 우선 완료하게끔 도커 설정을 전면 앞단에 배치하여 root권한을 제거



효과: 컨테이너 에스컬레이션 취약점 원천 봉쇄

인프라 보안 리스크 감소와 권한 문제로 인한 불필요한 컨테이너 장애 재발을 방지

```
tempo-init:
  image: alpine:latest
  user: "root"
  volumes:
    - ./config:/etc/tempo
    - tempo-data:/var/tempo/wal
  # 컨테이너 안의 tempo 유저에게 소유권을 넘겨주고 바로 종료됩니다.
  command: chown -R 10001:10001 /etc/tempo /var/tempo/wal
```

```
tempo:
  image: grafana/tempo:3.0.0
  container_name: tempo
  # user: "root"
  command: ["-config.file=/etc/tempo/tempo-config.yaml"]
  expose:
    - "3200" # Tempo HTTP API
```

4.4 고도화 ④: Grafana Alloy 마이그레이션

Alloy UI : <http://1.201.177.162:12345/>
grafana : <http://1.201.177.162:3000/>



배경: 수집기 난립에 따른 자원 부하 증가

데이터 수집을 위해 Promtail, Prometheus 등 수집 에이전트가 산발함, 또한 Grafana 는 Promtail 이용자들에게 Alloy로의 마이그레이션을 권장함



조치: Grafana Alloy 기반 파이프라인 마이그레이션

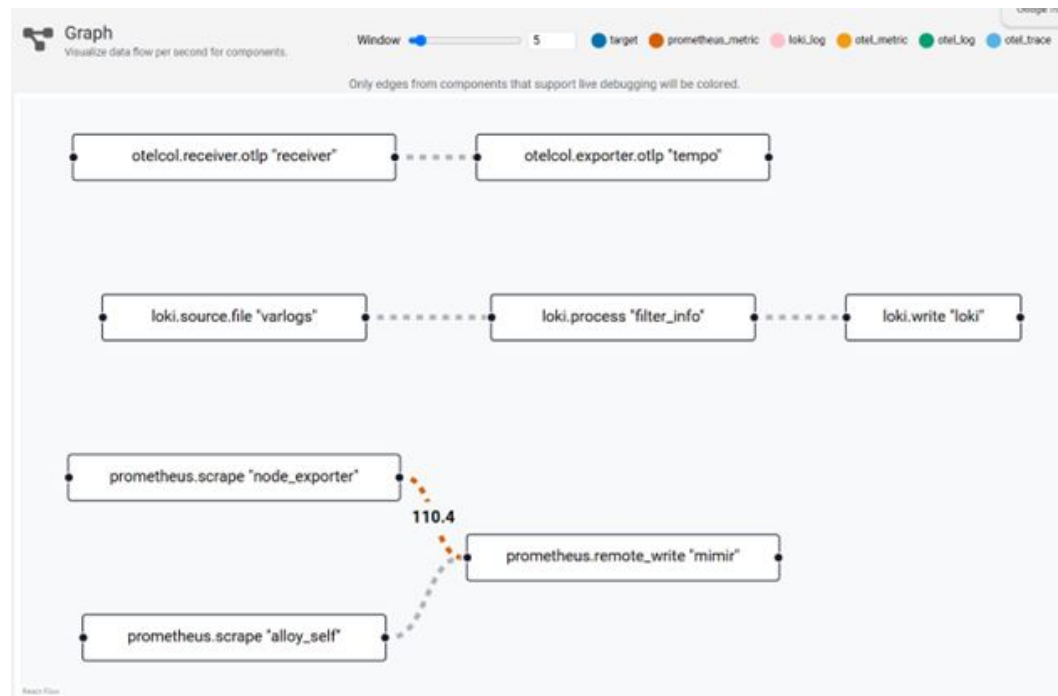
단일 다용도 에이전트인 `Alloy`를 사용하여 메트릭, log, trace 데이터 수집 전송을 통합



효과: 관리대상 축소

관리대상 축소에 따른 설정 불일치 리스크 감소 및 Alloy UI로 디버깅 가시성 향상

SERVICE	CREATED	STATUS
alloy	6 days ago	Up 6 days
grafana	6 days ago	Up 6 days
loki	6 days ago	Up 6 days
mimir	6 days ago	Up 6 days
minio	6 days ago	Up 6 days (healthy)
node-exporter	6 days ago	Up 6 days
tempo	6 days ago	Up 6 days



4.4 고도화 ④: Grafana Alloy 마이그레이션

Alloy UI : <http://1.201.177.162:12345/>
grafana : <http://1.201.177.162:3000/>



배경: 수집기 난립에 따른 자원 부하 증가

데이터 수집을 위해 Promtail, Prometheus 등 수집 에이전트가 산발함, 또한 Grafana 는 Promtail 이용자들에게 Alloy로의 마이그레이션을 권장함



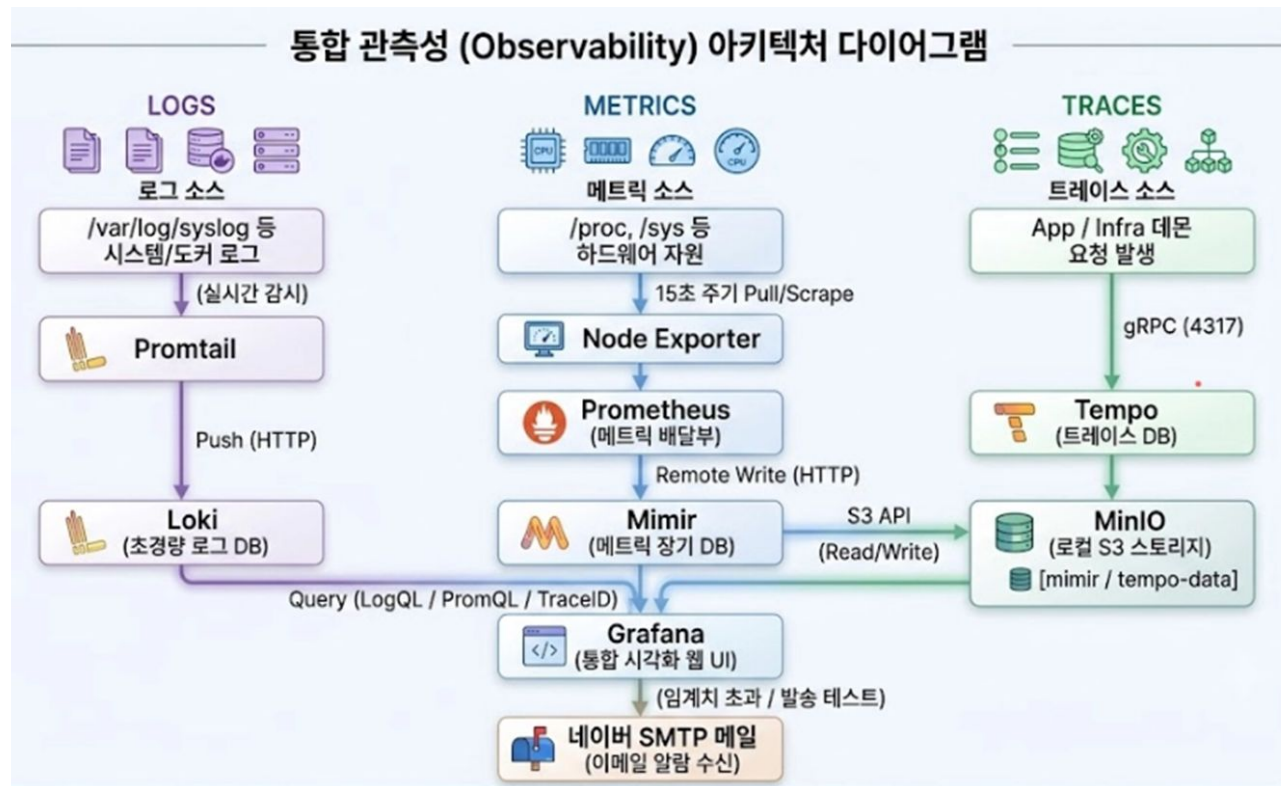
조치: Grafana Alloy 기반 파이프라인 마이그레이션

단일 다용도 에이전트인 `Alloy`를 사용하여 메트릭, log, trace 데이터 수집 전송을 통합



효과: 관리대상 축소

관리대상 축소에 따른 설정 불일치 리스크 감소 및 Alloy UI로 디버깅 가시성 향상



4.4 고도화 ④: Grafana Alloy 마이그레이션

Alloy UI : <http://1.201.177.162:12345/>
grafana : <http://1.201.177.162:3000/>



배경: 수집기 난립에 따른 자원 부하 증가

데이터 수집을 위해 Promtail, Prometheus 등 수집 에이전트가 산발함, 또한 Grafana 는 Promtail 이용자들에게 Alloy로의 마이그레이션을 권장함



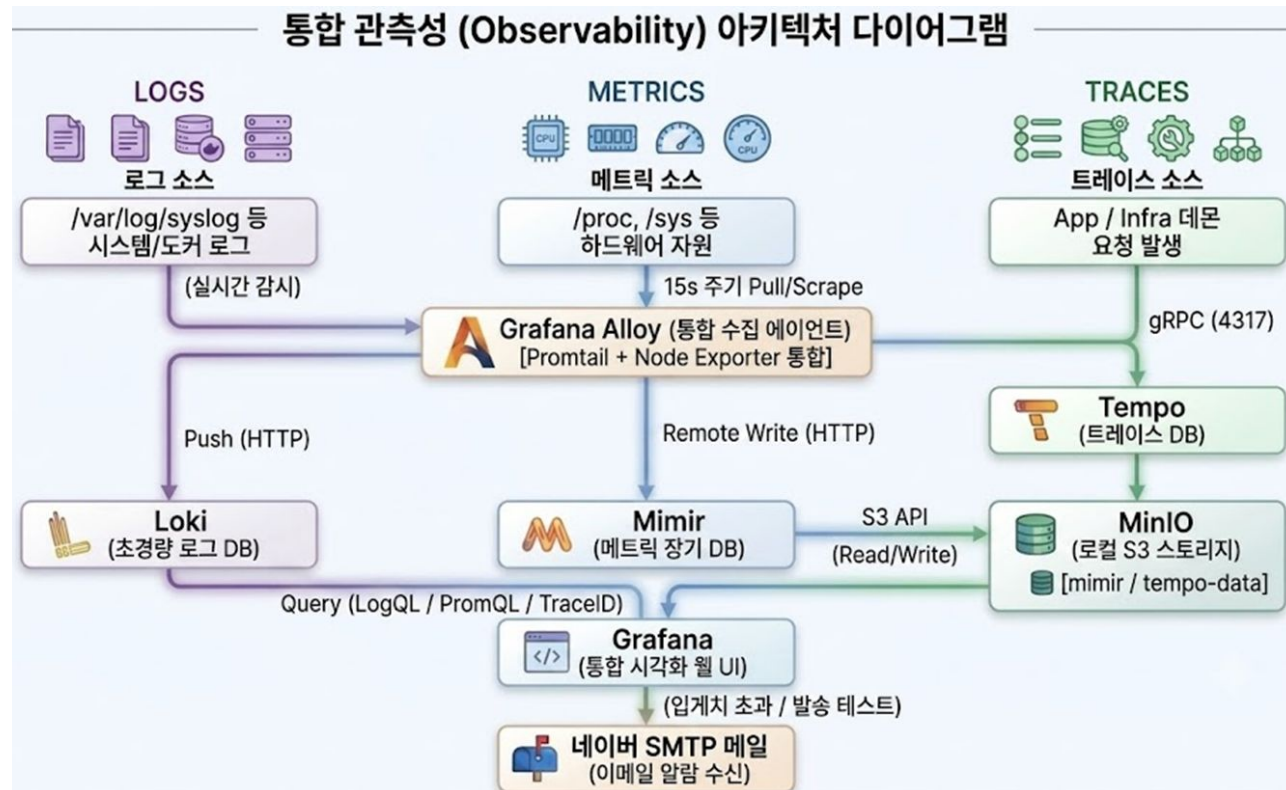
조치: Grafana Alloy 기반 파이프라인 마이그레이션

단일 다용도 에이전트인 `Alloy`를 사용하여 메트릭, log, trace 데이터 수집 전송을 통합



효과: 관리대상 축소

관리대상 축소에 따른 설정 불일치 리스크 감소 및 Alloy UI로 디버깅 가시성 향상



05. 트러블슈팅

5.1 트러블슈팅: MiniO 종속 컨테이너 기동 실패

🐛 현상 (Symptom)

`docker compose up` 명령으로 실행 시, Mimir 및 Tempo 컨테이너가 재시작 루프에 빠집니다.

🔍 원인 (Root Cause)

Docker compose의 `depend_on`은 시작 순서만 제어할 뿐, Ready를 보장하지 않습니다.

그에 따라 MinIO 내부 초기화 시간 중 종속 서비스 호출 시 실패가 발생합니다.

🔧 조치 및 해결 (Resolution)

MinIO 컨테이너 내부에 ``/minio/health/live`` healthcheck 루틴을 장착하고, 타 컴포넌트의 ``condition: service_healthy`` 구성을 선언하여 의존 기동을 보장했습니다.

```
healthcheck:
  test: ["CMD", "curl", "-f", "http://localhost:9000/minio/health/live"]
  interval: 5s
  timeout: 5s
  retries: 5
```

```
depends_on:
  minio:
    condition: service_healthy # Tempo도 MinIO가 완전히 켜진 후 실행되
```

SERVICE	CREATED	STATUS
grafana	8 hours ago	Up 8 hours
loki	8 hours ago	Up 8 hours
mimir	8 hours ago	Up 8 hours
minio	8 hours ago	Up 8 hours (healthy)
node-exporter	8 hours ago	Up 8 hours
prometheus	8 hours ago	Up 8 hours
promtail	8 hours ago	Up 8 hours
tempo	8 hours ago	Up 8 hours

5.2 트러블슈팅: Grafana 대시보드 연결 실패

현상 (Symptom)

데이터 소스 설정 화면에서 'Save & Test' 실패 시 정상(Success) 연동을 확인했으나, 실제 대시보드에는 'No Data'만 출력

원인 (Root Cause)

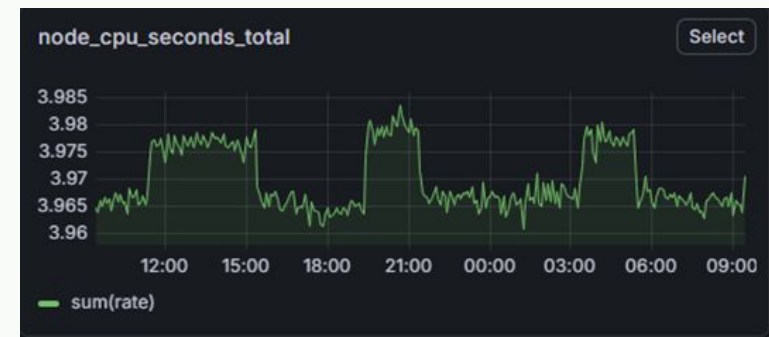
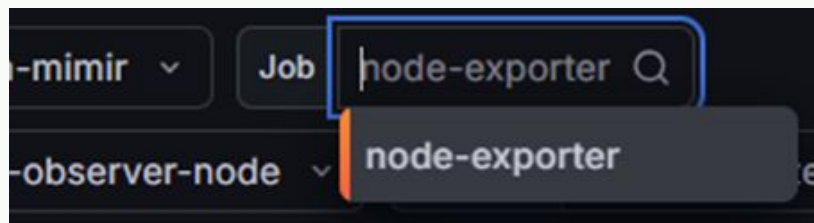
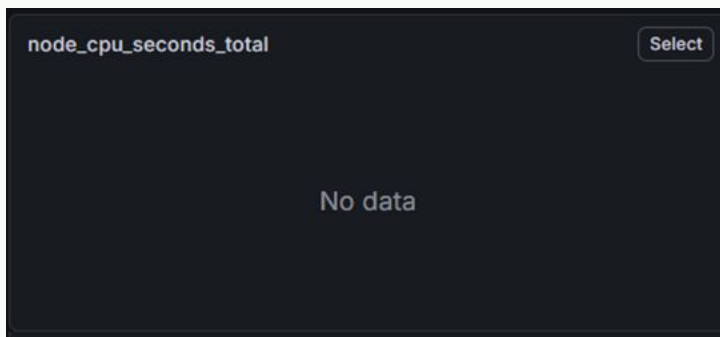
과거에 사용하던 레거시 Job 이름들이 대시보드에 잔존함을 확인.

패널 쿼리는 최신 Job이름을 바라보지 못함.

조치 및 해결 (Resolution)

대시보드 환경 설정 및 메트릭 명칭 정리.

수집기 측에서 더 이상 사용하지 않는 레거시 Job 삭제



5.3 트러블슈팅: Grafana 프로비저닝 실패

현상 (Symptom)

파일 기반 대시보드 프로비저닝 적용 시,
Json파일이 “유효하지 않은 파일”로
판단되어 자동 로드 차단 및 UI노출 실패

원인 (Root Cause)

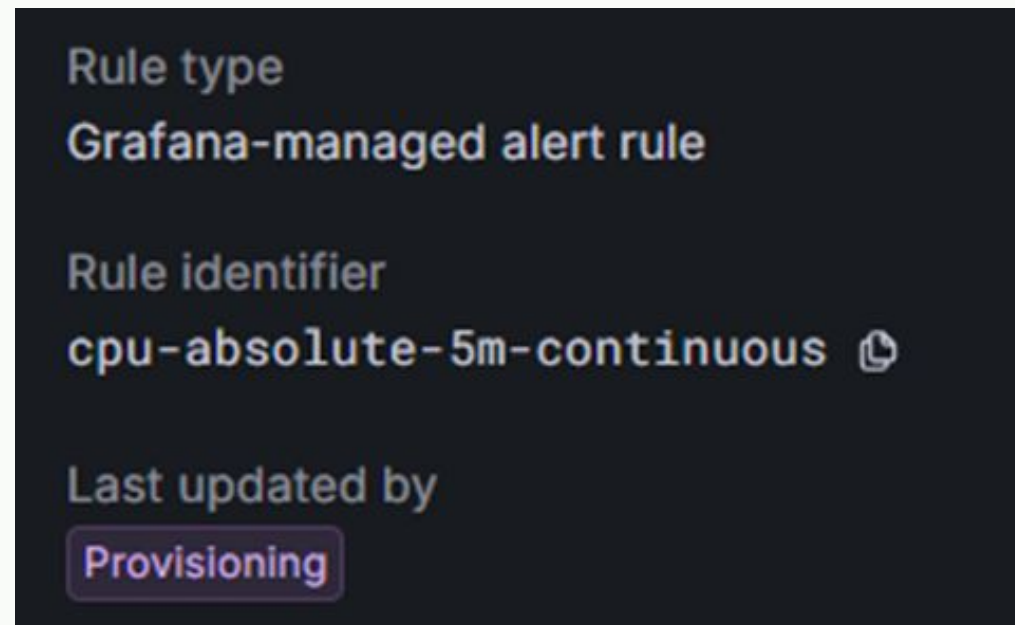
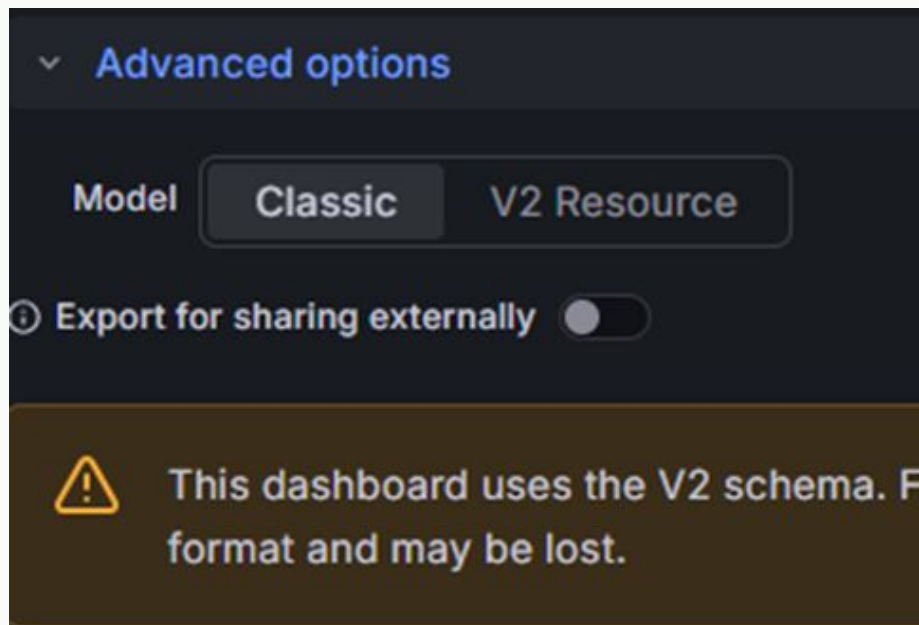
Grafana UI에서 Export한 대시보드가
V2 Resource 스키마로 추출됨

파일 프로비저닝은 해당 스키마 구조를
파싱하지 못함

조치 및 해결 (Resolution)

대시보드 Export시 Advanced
Options로 진입

Model을 Classic으로 전환 후, Raw
Json형태로 추출하여 폴더에 배치



5.4 트러블슈팅: Tempo v3.0 기동 실패

현상 (Symptom)

데이터 보존 정책(Retention) 추가 시
컨테이너 무한 재기동 발생

로그 확인 시 compactor_config 필드
파싱 불가 오류 발생

원인 (Root Cause)

Tempo 3.0로 변경되면서 Compactor
모듈이 backend_scheduler 와
backend_worker로 대체됨

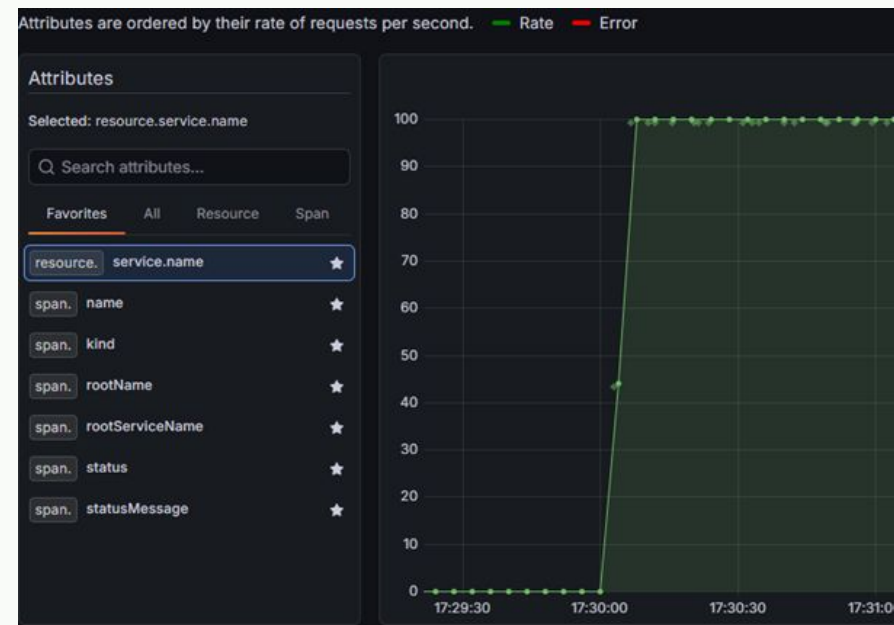
최신 버전이기에 레퍼런스 부존재 및
AI인식 오류

조치 및 해결 (Resolution)

공식문서에 따라 기존의
compactor설정을 제거하고
backend_worker구조 도입

compaction 및 block_retention을
정의하여 v3.0규격에 맞게 수정

```
backend_worker:  
  compaction:  
    block_retention: 72h  
    compacted_block_retention: 1h  
    compaction_window: 1h
```



06. 배운 점 및 한계점

6.1 프로젝트 수행을 통해 배운 점

Observability 3대 신호 연계

모니터링 지표로는 크게 **Log, Metric, Trace**가 있으며, 각 지표를 단독으로 활용할 때보다 **상호 연계** 시 더욱 큰 가치를 창출합니다.

특히 장애 대응 시 **MTTR(평균 복구 시간)**을 단축할 수 있음을 확인하였습니다.

품질 및 보안의 내재화

제한적인 단일 VM 환경 내에서도 **Healthcheck**와 **Least Privilege(최소 권한)** 원칙을 적용하였습니다.

또한 **자동화된 프로비저닝** 기법을 도입하여 시스템의 신뢰성과 품질을 극대화하는 역량을 확보하였습니다.

선 고정 버전 수립 전략

초기 설계 단계부터 기술스택의 **버전에 대한 이해화 고정**의 중요성을 느꼈습니다

이는 인프라의 **멥등성(Idempotency)** 과 운영 안정성을 확보하는 데 필수적인 요소임을 깨달았습니다.

6.2 프로젝트의 기술적 한계점



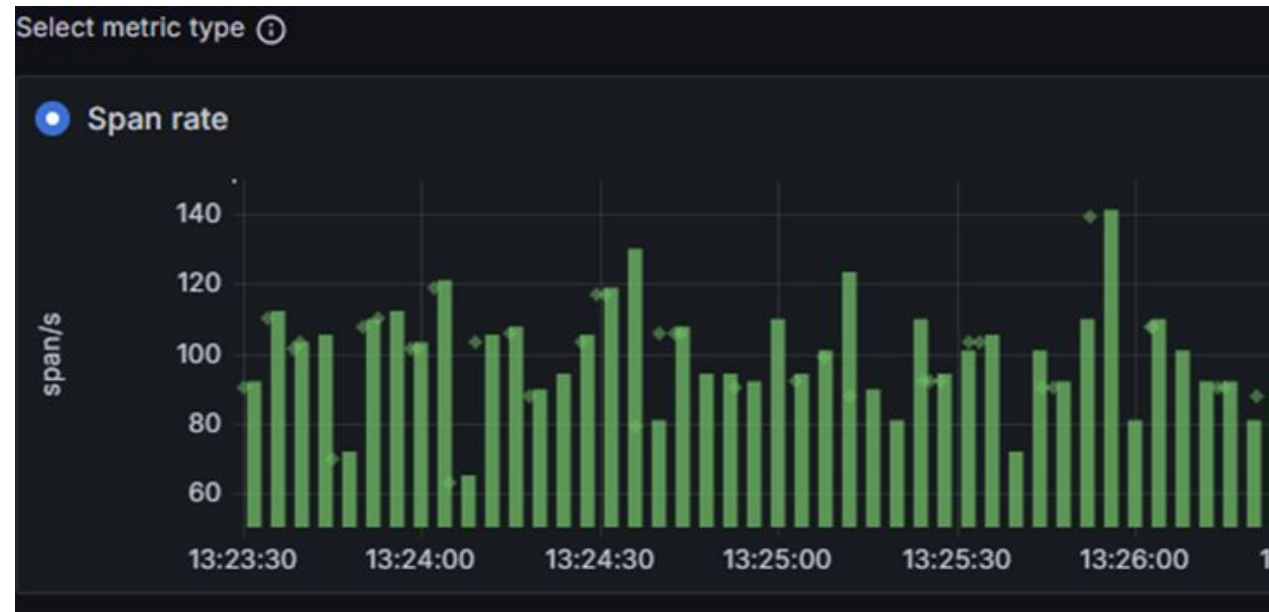
단일 VM 환경으로 인한 분산 제약

본 아키텍처는 단일 VM 및 Docker Compose 구조로 가동되므로 아키텍처의 MSA화, 분산 노드 배치나 고가용성을 실험하는 것에 명확한 환경적 제약이 따랐습니다.



애플리케이션 계층 연동의 부재

별도로 연동된 서비스가 제한적이어서 수집된 트레이스 데이터가 전적으로 시스템 및 인프라 측면의 기반 구축 위주로만 제한적 검증되었습니다.



07. 향후 발전

7. 향후 발전 (Next Steps)

☐ 모니터링 스케일 아웃

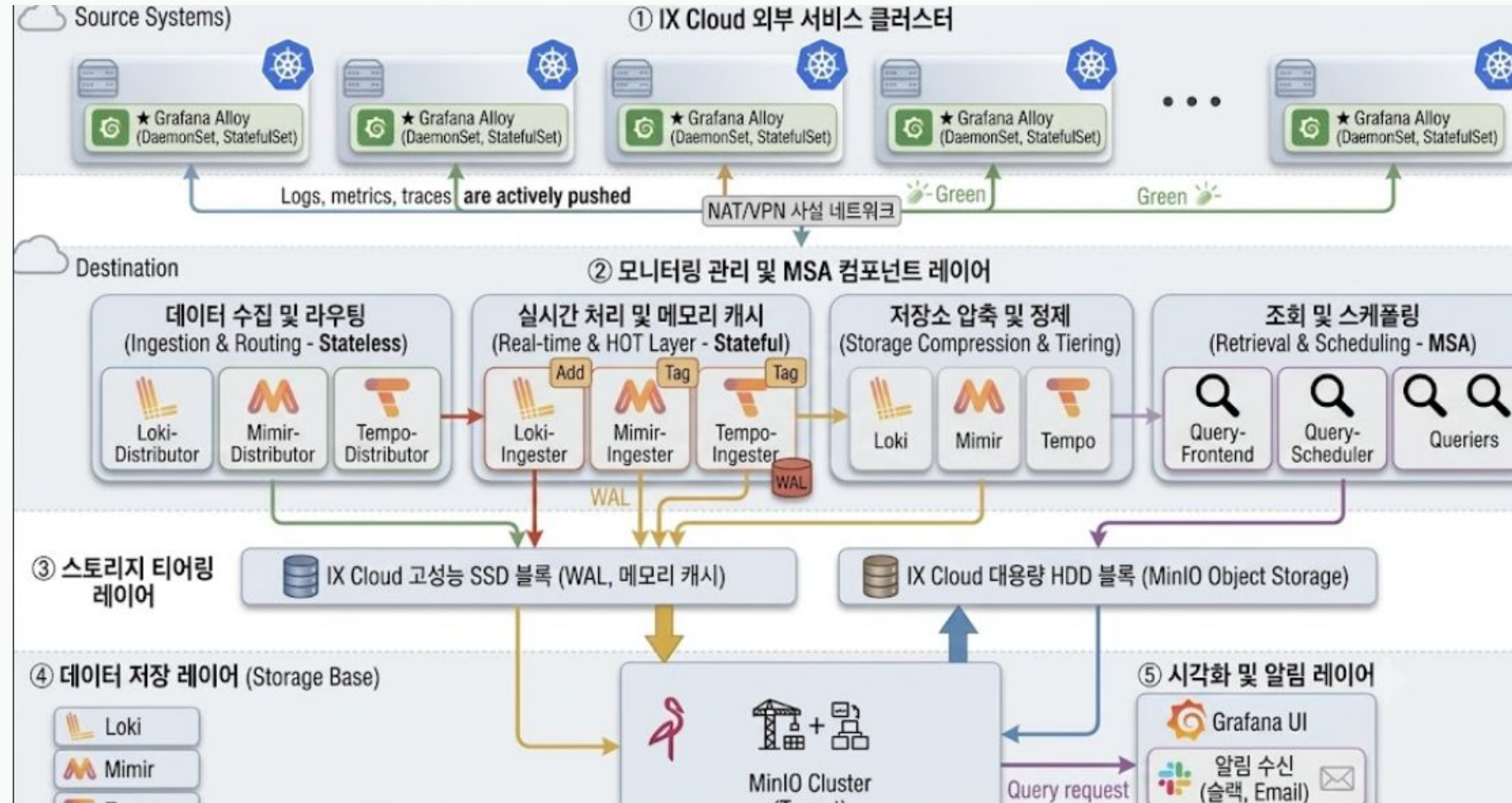
중규모화: K3s 기반 3~4개 멀티 노드 전개

📦 수집 컴포넌트 분산

MSA화 : 컴포넌트 모듈 단위로 분배하여
Microservices화

✳️ 모니터링 대상 연동

실제 관측 대상과의 연동: 내, 외부 서비스
클러스터 MSA 레이어와의 연동



Q & A

LGTM Observability Stack 프로젝트 발표를 마칩니다.

감사합니다.